CAS CS 538. Solutions to Problem Set 8

Due electronically via Gradescope, Monday April 6, 2026 11:59pm

Om Khadka, U51801771

How to submit: See instructions on Problem Set 1.

Trapdoor Hash Functions. For this problem, we will augment collision-resistant hashing with a useful property.

We will consider keyed collision-resistant hash functions that take a hashing key pk and two input values (which we will call (a, b)), and output one value. We will now augment these hash functions with an additional property: a trapdoor secret key sk that enables easy collision finding.

We formalize this property as follows. We add a key generation algorithm G that outputs a public hashing key pk and a trapdoor key sk. There is also an efficient algorithm T to find collisions given sk: specifically, $T(sk, a_1, b_1, a_2)$, outputs b_2 such that $H(pk, a_1, b_1) = H(pk, a_2, b_2)$. We still require that the hash function remains collision resistant for anyone who is given pk but not sk.

In the next two problems you will build trapdoor hash functions. Of course, if a hash function has a trapdoor, then the person who generated the key will be able to find collisions.

Problem 1. (30 points) Recall the DL-based family in Section 8.5.1 of the textbook. Let q be a Sophie Germain prime, so that $p = 2q + 1$ is also prime. Let $x, y \in QR_p$ and $a, b \in \{1, 2, \ldots, q\}$. The public key is (x, y) and $H((x, y), a, b) = \text{abs}(x^a y^b \bmod p)$, where $\text{abs}(z)$ returns z if $z < p/2$ and $-z$ otherwise.

Show that H is actually a trapdoor hash family. In other words, demonstrate an algorithm G creates $pk = (x, y)$ and some sk (your job is to figure out what sk should be), and algorithm T.

Solution. Given the abstract definition of the trapdoor hash func before Q1, this question is now basically making me show an actual example of how a such a func would work. The main way I will solve this is (basically) through modular arithmetic.

$\mathbb{G}$ and $\mathcal{T}$

Let $\mathbb{G}$ be the key gen for the trapdoor func, $\mathcal{T}$.

$\mathbb{G}()$:

- Select rnd $x \in QR_p \backslash \{1\}$.
- Select rnd $s \in \mathbb{Z}_p^*$.
- Compute y as: $y = x^s \mod p$.
- $pk = (x, y)$, $sk = s$.

$\mathcal{T}(sk, a_1, b_1, a_2)$:

- Parse sk as s.
- Computes b_2 as: $b_2 = b_1 + s^{-1}(a_1 - a_2)$.
- Return b_2.

Correctness

Recall that $H((x,y),a,b) = abs(x^a y^b \mod p)$. There is an important property of this abs func that I need to say first-off:

- Because $x, y \in QR_p$, and QR_p is multiplpicatively closed, then $\forall a, b, x^a y^b \in QR_p$.
- q is odd. That means $p \equiv_4 3$. Note that -1 can only be a quad. residue $\mod p \iff p \equiv_4 1$. Therefore, $-1 \notin QR_p$.
- Therfore, if $z_1, z_2 \in QR_p, abs(z_1) = abs(z_2) \iff z_1 \equiv_p z_2$. Basically, the case that $z_1 \equiv -z_2$ is impossible.
  - Therefore we can just reduce the equation we need to solve from $abs(\cdot) \equiv_p abs(\cdot)$ to just $x^{a_1} y^{b_1} \equiv_p x^{a_2} y^{b_2}$.

Firstly we can just sub y for $x^s \mod p$:

$$x^{a_1+sb_1} \equiv_p x^{a_2+sb_2}$$

Now, from $\mathcal{T}$, we get $b_2 \equiv_q b_1 + s^{-1}(a_1 - a_2)$. By just multiplying both sides w/ s:

$$sb_2 \equiv_q sb_1 + a_1 - a_2 \to a_1 + sb_1 \equiv_q a_2 + sb_2$$

Now, note that x, q are both prime, and that x has order q. That means that exponents of x are equivalent $\mod q$ iff the powers are equivalent modulo p:

$$x^{a_1+sb_1} \equiv_p x^{a_2+sb_2} \to a_1 + sb_1 \equiv_q a_2 + sb_2$$

This equation holds perfectly. It is also the exact same output as from $\mathcal{T}$, which is assumed to output the collision of b_2. Therfore, this shows how $H(pk, a_1, b_1) = H(pk, a_2, b_2)$ if you run $\mathcal{T}(sk, a_1, b_1, a_2)$.

Game

Now for the proof that $\mathbb{G}, \mathcal{T}$ holds, I will make an adversary game, with a MITM. The MITM, $\mathcal{B}$, will try to simluate a discrete log (DL) game for $\mathcal{A}$, which is playing against the CRHF.

Let $\mathcal{A}$ be an eff. adversary, that can find collisions within H, w/ advantage ϵ, when only given pk. $\mathcal{B}$ will then also be another efficent adv., solving the DL problem in QR_p with the same probability.

DL Challenge $\mathcal{B}$ firstly gets $(p, g, \mathcal{Y})$, where g generates QR_p and $\mathcal{Y} \in QR_p$. The goal is to find s such that $g^s \equiv_p \mathcal{Y}$.

Execution $\mathcal{B}$ will set $pk \to (g, \mathcal{Y})$. This distribution is the same as a valid public key, outputted by $\mathbb{G}$. Then, $\mathcal{B}$ will give pk to $\mathcal{A}$. Finally, $\mathcal{A}$ will output $(a_1, b_1), (a_2, b_2)$, such that they cause a collision when both plugged into H.

Extracting the Key By the properties of *abs* from before, we are just solving for:

$$g^{a_1}\mathcal{Y}^{b_1} \equiv_p g^{a_2}\mathcal{Y}^{b_2}$$

Rearrage the terms:

$$g^{a_1-a_2} \equiv_p \mathcal{Y}^{b_2-b_1}$$

Let $\triangle a = (a_1 - a_2)$, and $\triangle b = (b_2 - b_1)$. Then, we're just solving for:

$$g^{\triangle a} \equiv_p \mathcal{Y}^{\triangle b}$$

Since $(a_1, b_1) \neq (a_2, b_2)$, there no way that $\triangle a \equiv_q \triangle b \equiv_q 0$. Imagine if $\triangle b \equiv 0$. Then $g^{\triangle a} \equiv_p 1$. And since g has order q, this forces $\triangle a \equiv_p 0$ as well. This is impossible. So let's just say that $\triangle b \not\equiv_q 0$.

Now, since q is prime, then $\triangle b$ is invertible $\mod q$.Then, $\mathcal{B}$ computes $(\triangle b)^{-1} \mod q$ and gets:

$$s \to \triangle a \cdot (\triangle b)^{-1} \mod q$$

This satisfies $\mathcal{Y} \equiv_p g^s$, therefore this is a DL challenge. $\mathcal{B}$ only wins when $\mathcal{A}$ finds a collision. Since DL is hard, $\mathcal{A}$'s success is neglgible.

This proves the security of the construction of $\mathbb{G}, \mathcal{T}$. $\square$

Problem 2. (40 points at 10 each) In this problem, we will build a collision-resistant hash function based on the hardness of factoring rather on the discrete logarithm problem.

Let q_1 and q_2 be two distinct Sophie Germain primes, so that $p_1 = 2q_1 + 1$ and $p_2 = 2q_2 + 1$ are primes. Let $n = p_1p_2$. Denote by QR_{p_1}, QR_{p_2}, and QR_n the groups of squares modulo p_1, p_2, and n, respectively. We know that QR_{p_1} and QR_{p_2} are cyclic; thus, let $g_1 \in QR_{p_1}$ generate QR_{p_1} and $g_2 \in QR_{p_1}$ generate QR_{p_2}.

(a) Show by CRT (see notes from Discussion 8) that QR_n is cyclic (the easiest way to do this is to try to construct a generator of QR_n and show that it is indeed a generator by showing that its order is q_1q_2).

Solution. This solution really involves a lot with the CRT, alongside the properties of gcd/lcm's (in relation to quadratic residues).

Proof

By the Chinese Remainder Theorem (CRT), since $n = p_1p_2$ and $\gcd(p_1, p_2) = 1$, we can actually form a ring isomorphism of the following: $\mathbb{Z}_n \equiv \mathbb{Z}_{p_1} \times \mathbb{Z}_{p_2}$. Because of this, we implicity get the quadratic residue subgroups:

$$QR_n \equiv QR_{p_1} \times QR_{p_2}$$

The size of QR_{p_1} is q_1 and similarily for QR_{p_2}, it is q_2. And since q_1, q_2 are distinct primes, $\gcd(q_1, q_2) = 1$.

Also note, via the given g_i's I'm given, and by using CRT, it is possible to create a **distinct** element, $g \in \mathbb{Z}_n^*$, that satisfies the following:

$$\begin{aligned} g &\equiv g_1 \pmod{p_1} \\ g &\equiv g_2 \pmod{p_2} \end{aligned}$$

Since g_1, g_2 are quadratic residues mod their respective primes, their CRT combination, g, should then also be a quadratic residue $\mod n$. Thus, $g \in QR_n$.

Via the laws of direct product groups, the order of an element is the LCM of the orders of its components:

$$\text{ord}_n(g) = \text{lcm}\left(\text{ord}_{p_1}(g_1), \text{ord}_{p_2}(g_2)\right)$$

And because g_1, g_2 are generators, then $\text{ord}_{p_1}(g_1) = |QR_{p_1}| = q_1$ and $\text{ord}_{p_2}(g_2) = |QR_{p_2}| = q_2$. I can sub these in to get:

$$\text{ord}_n(g) = \text{lcm}(q_1, q_2)$$

And since $\gcd(q_1, q_2) = 1$, it is then the case that $\text{lcm}(q_1, q_2) = q_1 q_2$. Therefore, $\text{ord}_n(g) = q_1 q_2$.

End

Because QR_n has the cardinality of $|QR_{p_1}| \cdot |QR_{p_2}| = q_1 q_2$, this proves how I can make an element, $g \in QR_n$, whose order $=$ the group size. Note that this element is a generator. Thus, QR_n is cyclic. $\square$

(b) Show how to efficiently factor n given $q_1 q_2$. In fact, you can factor n given any multiple of $q_1 q_2$, but we won't show it here. Hint: treat q_1 and q_2 as unknowns. Construct a system of equations for these unknowns and solve it.

Solution. The solution to this involves the vars I'm given: the pub. modulus n, and the value $Q = q_1 q_2$. From this, I can factor n out to its prime components, p_1, p_2.

Deriving the System of Equations

Firstly, expand the definition of n using the given relationship, $p_i = 2q_i + 1$:

$$\begin{aligned} n &= p_1 p_2 \\ &= (2q_1 + 1)(2q_2 + 1) \\ &= 4q_1 q_2 + 2q_1 + 2q_2 + 1 \\ &= 4(q_1 q_2) + 2(q_1 + q_2) + 1 \end{aligned}$$

Now, let $S = q_1 + q_2$ be the sum of the unknown primes. I can sub in the known value, $Q = q_1 q_2$, into the equation, getting:

$$n = 4Q + 2S + 1$$

Now solving for S:

$$2S = n - 1 - 4Q \implies S = \frac{n - 1 - 4Q}{2}$$

Solving the Unknowns

This gives us a system of two equations for the two unknowns so far, q_1, q_2:

1. $q_1 q_2 = Q$ (Product)
2. $q_1 + q_2 = S$ (Sum)

Note that, if two numbers have a sum S and product Q, then they are simply the roots of the quadratic equation:

$$x^2 - Sx + Q = 0$$

Now it is possible to solve for x, using the quadratic formula:

$$x = \frac{S \pm \sqrt{S^2 - 4Q}}{2}$$

And since q_1, q_2 are distinct primes, then their discriminant, $S^2 - 4Q$, should then also be positive. By computing these roots, you get $\{q_1, q_2\}$.

End

Once you've computed q_1, q_2 , you can compute the factors of n by just subbing in q_1, q_2 into the OG equation:

$$p_1 = 2q_1 + 1 \quad \text{and} \quad p_2 = 2q_2 + 1$$

Since all I've done is only basic arithmetic and int square roots (which is efficient, by polynomial in $\log n$), this proves how one can efficiently factor n. $\square$

(c) Let k be the bit length of n. Let g be a generator of QR_n. The public key for our hash function is n and g. The inputs will be two values in the set $\{1, 2, \ldots, q_1 q_2\}$. Let $H(a, b) = g^{a2^k + b} \bmod n$ (note that $a2^k + b$ is just concatenation of a and b as bit strings). Show that this is a collision-resistant hash family under the assumption that such n are hard to factor. Specifically, show how, given (a_1, b_1) and (a_2, b_2) that collide, one can factor n. You may use without proof the fact that n can be efficiently factored given any non-zero multiple of $q_1 q_2$.

Solution. I can prove the OG claim with a adv. game. Let $\mathcal{A}$ be some eff. adv., that outputs a collision $(a_1, b_1) \neq (a_2, b_2)$ with some non-negligible probability. I can then use this collision to factor n.

Getting E

By definition of a collision:

$$H(a_1, b_1) = H(a_2, b_2) \implies g^{a_1 2^k + b_1} \equiv g^{a_2 2^k + b_2} \pmod{n}$$

Now 2(a) shows us how g generates QR_n, and therefore how $ord_n(g) = |QR_n| = q_1 q_2$. Now note, in a cyclic group, $g^X \equiv g^Y \pmod{n} \iff X \equiv Y \pmod{ord_n(g)}$. Thus:

$$a_1 2^k + b_1 \equiv a_2 2^k + b_2 \pmod{q_1 q_2}$$

Now let $\Delta a = a_1 - a_2$ and $\Delta b = b_1 - b_2$. By rearranging the yields now:

$$\Delta a \cdot 2^k + \Delta b \equiv 0 \pmod{q_1 q_2}$$

This now implies that such an int, $E = \Delta a \cdot 2^k + \Delta b$ is a multiple of $q_1 q_2$.

E is not 0

Now note, the inputs satisfy $a, b \in \{1, \ldots, q_1 q_2\}$, so then $|\Delta a| < q_1 q_2$ and also $|\Delta b| < q_1 q_2$. Since k is the bit length of n, we then get $2^k > n > q_1 q_2$.

If $E = 0$, then $|\Delta a| \cdot 2^k = |\Delta b|$. If $\Delta a \neq 0$, the left side should be $\geq 2^k > q_1 q_2$, while the right side should be $< q_1 q_2$. This is a contradiction. Thefore $\Delta a = 0$, which then ALSO forces $\Delta b = 0$. This then form another contradiction, as $(a_1, b_1) \neq (a_2, b_2)$.

So then therfore, it must be the case that E is a *non-zero* int multiple of $q_1 q_2$.

Factoring

Not, we're given that n can be eff. factored, when provided with any non-0 multiple of q_1, q_2. Now, given $\mathcal{A}$'s collision, it is possible to compute $E = (a_1 - a_2)2^k + (b_1 - b_2)$, and then pass it to the factoring algorithm. This will then get the prime factors p_1, p_2 of n with the SAME PROBABILITY that $\mathcal{A}$ finds a collision.

End

So then, if finding collisions is non-neglibible, then it must be the case that finding n should also be non-neglibible. By going back to the assumption that factoring products of distinct safe primes is hard, then the success probability of $\mathcal{A}$ must therefore be negligible.

This proves that H is a collision-resistant hash family. $\square$

(d) Show also that this is a trapdoor hash family (like in Problem 1, you need to find what trapdoor information G should output and what the algorithm T will do).

Solution. The whole premesis of this solution kinda builds off of what I did in 2(c). Basically, an adv. can only break the function *only if they know* $q_1 q_2$. This is equivalent to have that secret key sk. I'll just prove how obtaining sk is just as hard as normally factoring n.

I'll use the construction from Problem 2(c), but now with a key gen. algo., G, and a collision-finding algo, T, that uses a trapdoor.

Functions

$\mathbb{G}()$

- Gets rnd and unique Sophie Germain primes q_1, q_2, and then computes safe primes $p_1 = 2q_1 + 1$, $p_2 = 2q_2 + 1$.

- Computes $n = p_1 p_2$ and the group order $Q = q_1 q_2 = \frac{(p_1-1)(p_2-1)}{4}$.
- Chooses a generator g of QR_n (via CRT, as in 2(a)).
- Now, let k be the bit length of n.
- Then, output $pk \leftarrow (n, g)$ and $sk \leftarrow Q$.

$\mathcal{T}(sk, a_1, b_1, a_2)$:

- Parses sk as Q.
- Calculates $b_2 \leftarrow \big(b_1 + (a_1 - a_2) \cdot 2^k\big) \bmod Q$.
- Returns b_2.

Correctness

This is checking that T outputs b_2, such that $H(a_1, b_1) = H(a_2, b_2)$. Now note, $H(a, b) = g^{a2^k+b} \bmod n$. A collision requires:

$$g^{a_1 2^k + b_1} \equiv g^{a_2 2^k + b_2} \pmod{n}$$

Note that g generates QR_n, $ord_n(g) = Q$. From cyclic group properties, $g^X \equiv g^Y \pmod{n} \iff X \equiv Y \pmod{Q}$. Therefroe, we need:

$$a_1 2^k + b_1 \equiv a_2 2^k + b_2 \pmod{Q}$$

Solving for b_2:

$$b_2 \equiv b_1 + (a_1 - a_2) 2^k \pmod{Q}$$

Note that this algo, T, computes exactly this value. So then, just subbing this b_2 back into the exponent congruence satisfies the condition, assuring that $H(a_1, b_1) = H(a_2, b_2)$.

End

The hash family remains collision-resistant for anyone, given ONLY $pk = (n, g)$. As shown in 2(c), finding a collision without knowing Q will allow one to eff. factor n. Under the assumption that factoring is hard, an adv.'s success probability is negligible. However, the key generator holds $sk = Q$, allowing them to bypass the hardness assumption, and to just compute colliding inputs b_2 via simple modular arithmetic in polynomial time.

Therefore, this proves that H is a valid trapdoor hash family. $\square$

Problem 3. (30 points) Show how to compute the root of an n-leaf Merkle tree in $O(\log n)$ space. More precisely, assume n is a power of 2. Given data items $x_1, x_2, \ldots, x_n$ as values to be placed in the leaves of the tree, and a hash function H, write pseudocode to compute the root r of the tree while never storing more than $2 + \log n$ hash values. Analyze the storage space used by your pseudocode to prove that is never more than $2 + \log n$. Note that we do not count the storage of the data items.

Solution. Note that, in the construction of this algo, each level, j, should only have 2^j leaves. This fact will be used later on in the explanation of my algo + analysis.

Algorithm Specification

Let $L = \log_2 n$. Now initialize an array $\texttt{S}[0 \ldots L]$ to $\perp$ (empty). Then, the hash function $H(\cdot, \cdot)$ will take two inputs.

1. **Init:** $\texttt{S}[j] \leftarrow \perp$ for all $j \in \{0, \ldots, L\}$.
2. **Process Leaves:** For $i = 1$ to n:
   - $\texttt{curr} \leftarrow H(x_i)$
   - $\texttt{level} \leftarrow 0$
   - **While** $\texttt{S}[\texttt{level}] \neq \perp$:
     - $\texttt{curr} \leftarrow H(\texttt{S}[\texttt{level}], \texttt{curr})$
     - $\texttt{S}[\texttt{level}] \leftarrow \perp$
     - $\texttt{level} \leftarrow \texttt{level} + 1$
   - $\texttt{S}[\texttt{level}] \leftarrow \texttt{curr}$
3. **Returns:** $\texttt{S}[L]$ as the Merkle root.

Space Complexity

The algo shouldn't ever store more than $2 + \log n$ hash values. I'll prove this below:

- **Array Storage:** $\texttt{S}$ has $L + 1 = \log_2 n + 1$ slots. Note that, by construction, each slot will hold AT MOST one hash value. Therefore, $\texttt{S}$ uses $\leq \log_2 n + 1$ hashes.
- **Temp Storage:** The vars $\texttt{curr}$ only holds exactly 1 hash value during calculation.
- **Max Storage:** During the $\texttt{While}$ loop, $\texttt{S}[\texttt{level}]$ and $\texttt{curr}$ are BOTH held in memory to compute their parent. And right after hashing, $\texttt{S}[\texttt{level}]$ is cleared. Therefore, the max simultaneous hash storage is:
$$|\texttt{S}|_{\max} + |\texttt{curr}| = (\log_2 n + 1) + 1 = \log_2 n + 2$$
- Int vars ($\texttt{i}$, $\texttt{level}$) don't count towards hash storage.

Now, since $\log_2 n + 2 \leq 2 + \log n$ for $n \geq 1$, the algo satisfies the $O(\log n)$ space constraint, while correctly computing the Merkle tree root. $\square$